ChatApp 컨셉 가이드

Oct 2, 2026 · @성준

개요

ChatApp은 C++ IOCP 기반 채팅 서버와 C# WinForms 클라이언트로 구성된 TCP 메신저 프로젝트다. 로비·채팅방, 말풍선 채팅, 파일 첨부, 프로필 갤러리, 방 관리까지 실제 메신저에 준하는 기능을 갖추되, 서버 쪽은 고성능 비동기 네트워크 서버가 실무에서 쓰는 패턴(IOCP, 커넥션 풀, Redis 캐시, MySQL 영속화)을 그대로 적용한다.

핵심 컨셉

백엔드/서버 개발자가 비동기 네트워크 서버의 실전 설계 패턴을 직접 손으로 구현하며 익히는 것이 이 프로젝트의 중심 목적이다. 커넥션 풀, 비동기 DB 요청 큐, Redis를 통한 실시간 카운터, 프로토콜 확장(페이지네이션·요청 ID) 같은 서버 패턴을 다루되, 그 결과물이 카카오톡처럼 친숙한 UX로 바로 체감되도록 클라이언트까지 함께 완성한다.

- 서버 아키텍처 학습이 1차 목표, 메신저 UI는 그 결과를 검증하고 보여주는 수단
- 사내·사설 환경에서 쓰는 걸 전제로 한 데모/학습 프로젝트 — 공인망 노출용 보안 경화는 범위 밖

주요 기능

기능	설명
로그인·프로필	로컬 계정 파일(토큰) 기반 재접속, 닉네임 변경, 프로필 이미지(URL·업로드·로컬 파일)
로비·채팅방	방 동적 생성·삭제·이름변경, 방장 자동 위임, 방 프로필 이미지
채팅	말풍선 UI, 파일 첨부(진행률 표시), 링크 미리보기, 메시지 삭제
방 목록	전체 방 / 내 방 탭, 페이지네이션, 이름 검색
방 멤버 목록	현재 접속자 / 전체 참여 이력 탭, 유저별 접속 상태
프로필 갤러리	여러 장 업로드, 대표 이미지 지정, 삭제
스킨	교체 가능한 색상 테마
시스템 로그	연결·로그인·오류 등 이벤트 로그, 레벨별 색상 구분

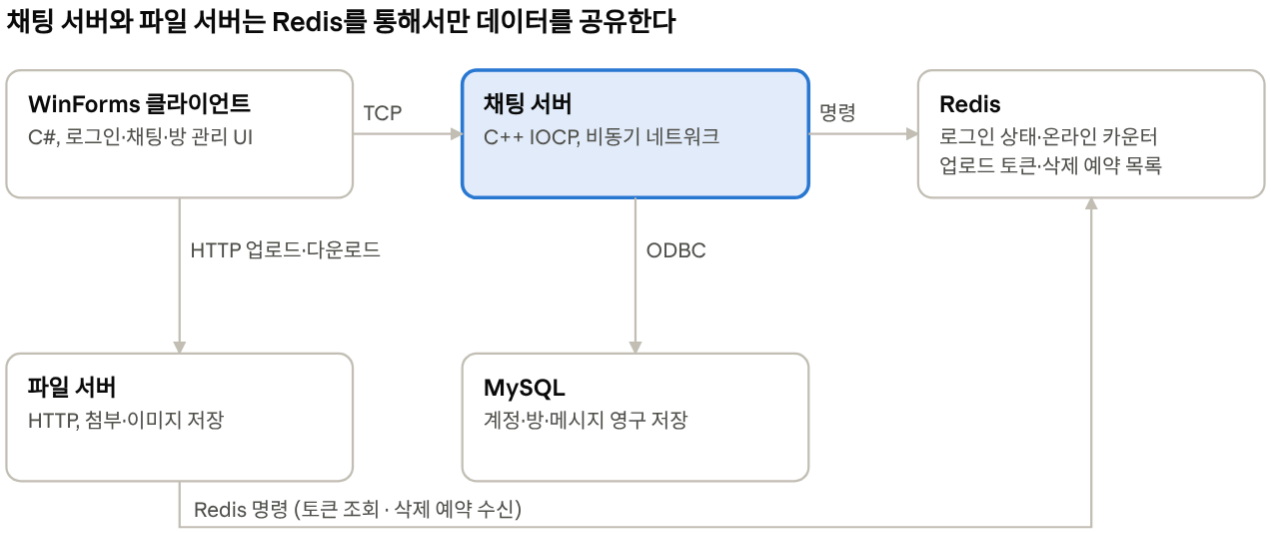
UI/디자인 컨셉

카카오톡 스타일을 기본 레퍼런스로 삼는다. 내가 보낸 메시지는 우측 정렬(노란 계열 말풍선), 상대는 좌측 정렬(흰색)로 구분하고, 프로필 이미지는 원형으로 표시한다.

- **카드형 레이아웃** — 각 영역(서버 접속, 채팅방, 시스템 로그)을 둥근 모서리 흰 카드로 감싸 구획을 나눈다.
- **탭 구성** — 대화·채팅방·갤러리·시스템 로그 네 개 탭으로 화면을 분리한다. 네이티브 탭 컨트롤의 렌더링 한계 때문에 탭 헤더는 자체 구현한 버튼으로 대체했다.
- **스킨 시스템** — 액센트 색상을 교체하면 버튼·테두리·말풍선 색이 함께 바뀌는 테마 체계. 창/페이지 배경은 가독성을 위해 스킨과 무관하게 고정된 밝은 색을 유지한다.

아키텍처 개요

클라이언트와 서버는 TCP 커스텀 프로토콜로 통신한다. 서버는 MySQL(계정·방·메시지 영구 저장)과 Redis(로그인 상태·온라인 카운터 등 실시간 데이터)를 함께 쓰고, 첨부파일·프로필 이미지는 별도 파일 서버가 HTTP로 처리한다.



전체 구조 · 클라이언트·채팅 서버·파일 서버·MySQL·Redis

채팅 서버는 클라이언트(TCP)·MySQL(ODBC)·Redis와 연결된다. 파일 서버는 채팅 서버와 직접 통신하지 않고, 클라이언트의 HTTP 요청과 Redis(업로드 토큰·삭제 예약)만 다룬다.

핵심 구현 포인트

네트워크 I/O

- IOCP 비동기 모델 — `AcceptEx` / `ConnectEx` 로 연결을 맺고, 이후 모든 송수신은 `WSARecv` / `WSASend` 완료 통지로만 진행된다. 세션당 recv 1개, send 1개까지만 동시에 진행 중인 것을 전제로 설계했다.
- 진행 중 I/O 카운트(`_pendingIoCount`) — 진행 중인 Recv·Send·DisconnectEx 완료 통지가 모두 돌아오고 종료 요청도 끝난 시점에만 세션 정리를 마무리한다. 종료 통지(OnDisconnected)는 CAS 가드로 한 번만 호출된다.
- 수신 링버퍼(`CRingBuffer`) — 버퍼 끝에서 앞으로 되감기는(wrap) 구조라, 경계를 넘는 데이터는 최대 2개의 연속 구간(`WSABUF[2]`)으로 WSARecv에 넘기고, 한 덩어리로 읽어야 할 때는 선형으로 복사해(Peek) 읽는다. 송신은 링버퍼 대신 청크 기반 `CSendBuffer` 를 모아 scatter-gather로 한 번에 WSASend한다.

데이터 계층

- DB 커넥션 풀 — ODBC 커넥션을 락프리로 대여·반납하고, 끊긴 커넥션은 백그라운드 스레드가 주기적으로 재연결을 시도한다. DB 요청은 별도 큐에 쌓여 비동기로 처리된다.
- Redis 비동기 클라이언트 — 명령 전송도 콜백 기반이며, 일시적 연결 오류는 지정 횟수만큼 지연을 두고 재시도한다. 로그인 상태(`User:{id}`), 방 채팅 기록(`RoomChatOrder:{roomId}` 등), 방 멤버 집계(`RoomMembers:{roomId}`)처럼 자주 바뀌는 실시간 데이터는 Redis에, 계정·방·메시지처럼 영구히 남아야 하는 데이터는 MySQL에 둔다.
- 방 입장 이력 집계 — 입장할 때마다 MySQL에 기록을 남기는 것과 별도로 Redis Set에 사용자 ID를 추가(SADD)해두면, 방 목록에 표시할 "참여 인원수"를 매번 DB를 조회하지 않고 집합 크기(SCARD)로 즉시 구할 수 있다.

프로토콜 설계

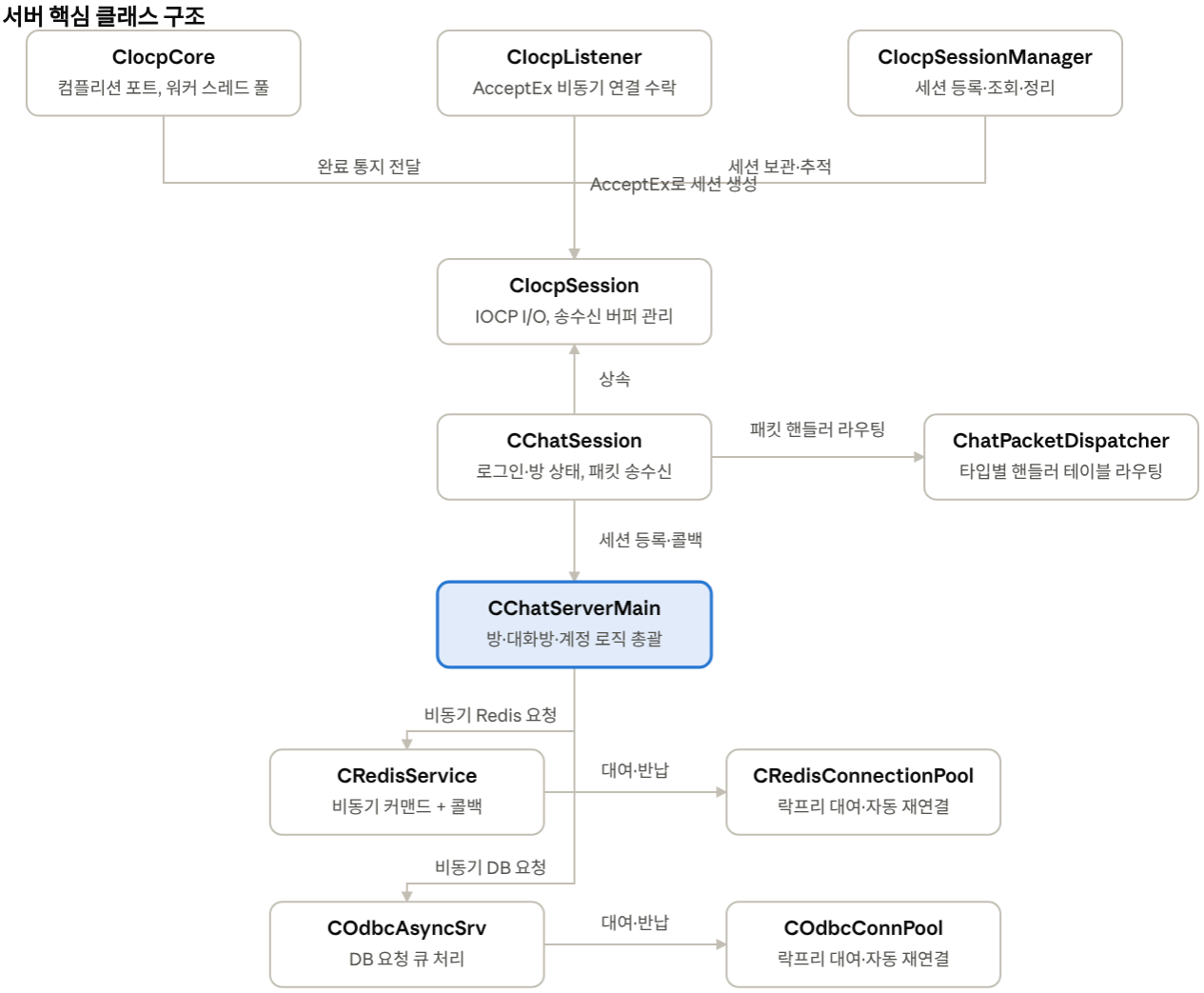
- 고정 헤더 + 가변 바디 — 모든 패킷은 `#pragma pack(1)` 로 패딩 없이 정렬한 `{size, type}` 헤더 뒤에 패킷별 바디가 붙는 구조다.
- 목록형 요청의 공통 패턴 — 방 목록, 방 멤버 목록처럼 여러 건을 돌려주는 요청은 `requestId` (응답과 요청을 짝짓는 식별자), `page` , `pageSize` 를 공통으로 쓰고, 항목을 하나씩 스트리밍한 뒤 완료 응답으로 마무리하는 동일한 흐름을 따른다.

동시성 보호

- 연결별 상태 초기화 — 세션 객체는 `AcceptEx` 슬롯마다 미리 만들어 두고, 새 연결이 완료될 때마다 송수신 큐·카운터 같은 연결별 상태를 명시적으로 리셋한다.
- 원자적 카운터로 레이스 방지 — 연결 종료 통지가 여러 스레드에서 동시에 들어올 수 있는 지점은 `std::atomic` 과 CAS 기반 가드로 한 번만 처리되도록 막는다.

서버 클래스 구조

CChatSession이 ClocpSession을 상속해 IOCP 송수신을 담당하고, 패킷은 디스패처가 타입별 핸들러로 라우팅한다. 로직은 전부 CChatServerMain에 모이고, 거기서 Redis와 DB는 각각 비동기 서비스 계층(CRedisService, COdbcAsyncSrv)을 거쳐 커넥션 풀에 접근한다.



서버 클래스 구조 · IOCP 엔진이 세션을 만들고 CChatServerMain이 Redis·DB를 조율

세션 → 서버 → Redis/DB로 이어지는 호출 방향이 곧 책임의 경계다: 세션은 네트워크만, 서버는 로직만, 서비스 계층은 커넥션 관리만 맡는다.

네트워크 엔진 구조

네트워크 계층은 엔진(IOCP, RIO)에 독립적인 공통 기반 위에 쌓여 있다. CNetworkFactory가 IOCP 코어 또는 RIO 코어(EngineCoreRef)를 받아 std::visit로 해당 엔진의 서버·클라이언트 서비스와 세션을 만들고, CNetService와 CSession이 두 엔진이 공유하는 계약을 정의한다.



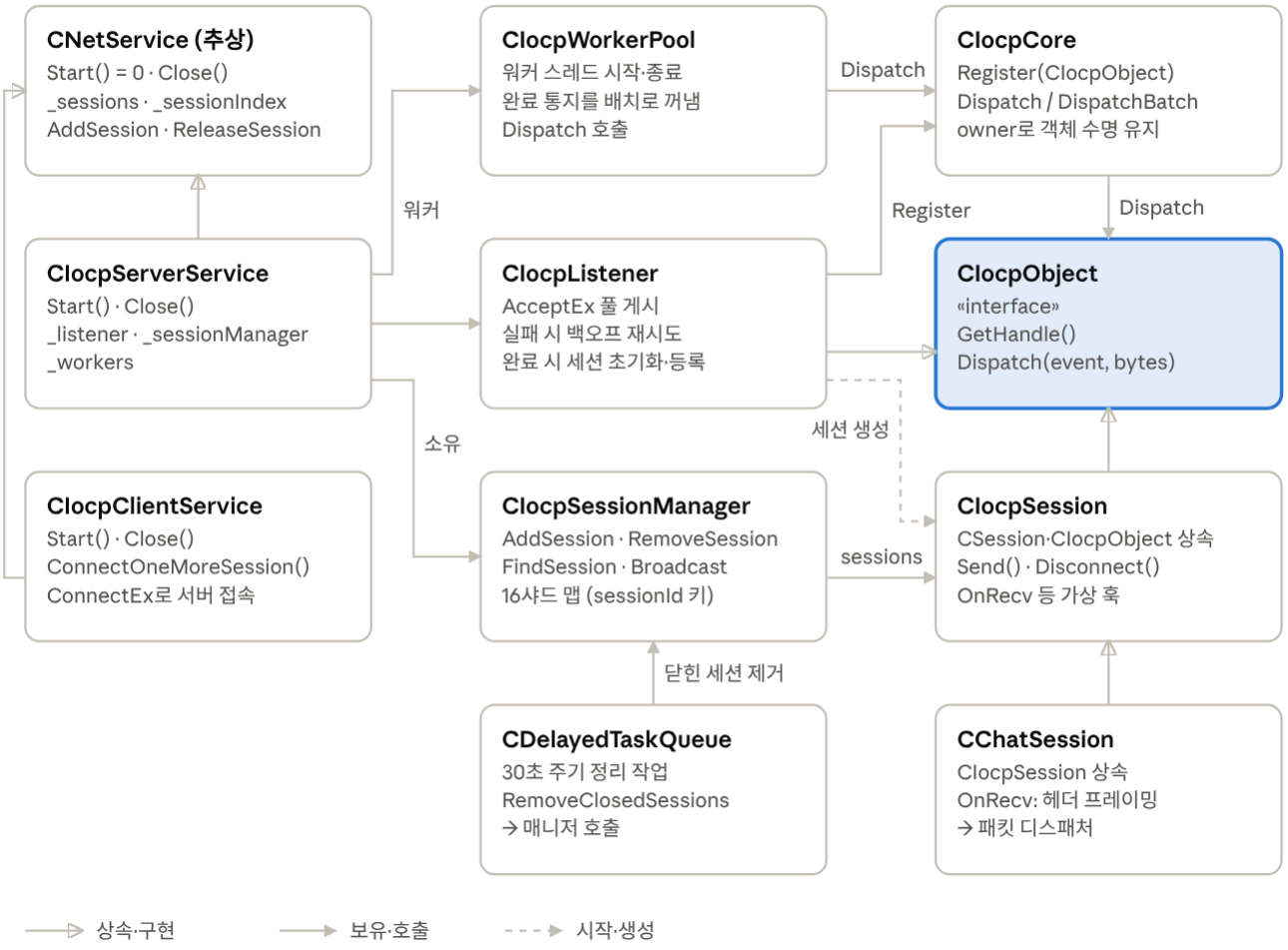
엔진 추상화 계층 · 팩토리, 서비스·세션 기반, IOCP/RIO 구현

CNetService는 세션 생성·추적과 종료 대기(Close)를, CSession은 세션 ID·상태·송수신 훅을 맡는다. 확장 함수(AcceptEx, ConnectEx, DisconnectEx)는 CSocketUtils가 로드하고, 수신 데이터는 CRingBuffer가 쌓는다.

IOCP 객체 구성

IOCP 쪽은 객체를 코어에 등록해 두면 완료 통지가 왔을 때 코어가 그 객체의 Dispatch()를 호출한다는 한 가지 규칙으로 묶인다. 리스너와 세션이 모두 ClocpObject를 구현하고, 서버 서비스가 워커 풀·리스너·세션 매니저를 소유한다.

서비스가 워커 풀·리스너·매니저를 쥐고, 객체는 코어에 등록된다



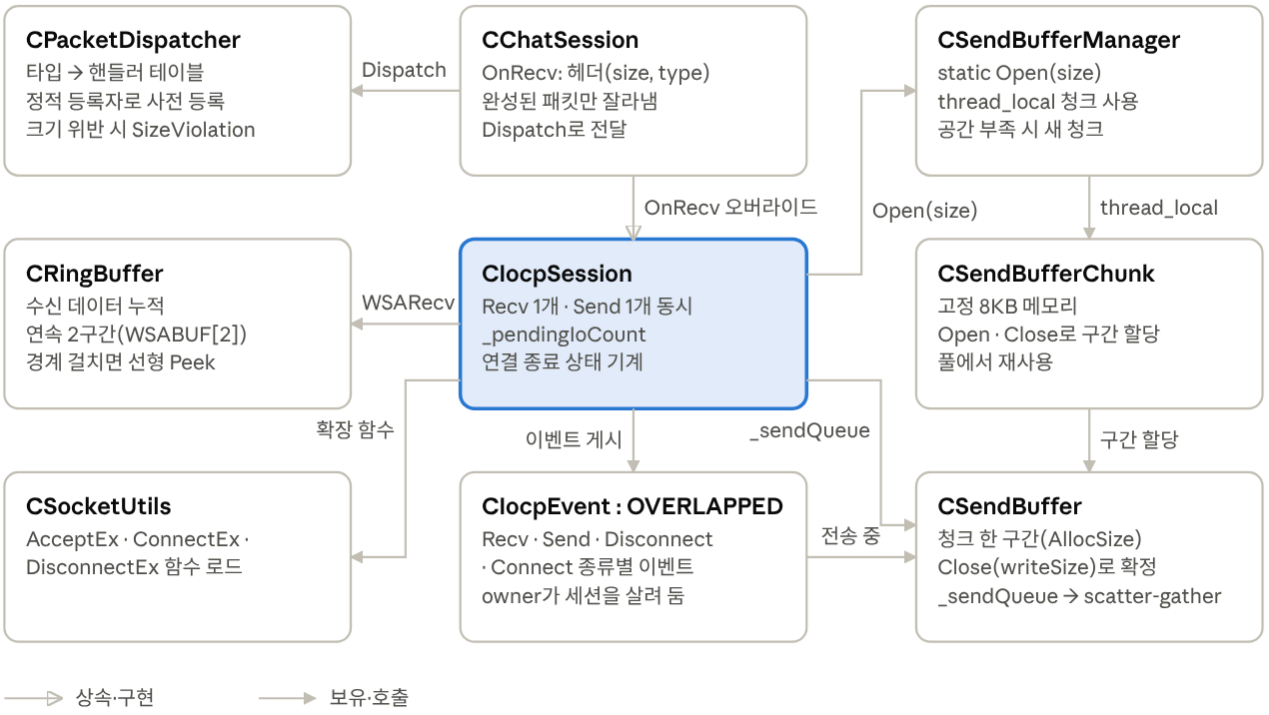
IOCP 클래스 구조 · 서비스·코어·리스너·세션·세션 매니저

채팅 세션(CChatSession)은 ClocpSession을 상속해 OnRecv에서 헤더 단위로 패킷을 잘라 디스패처에 넘긴다. 클라이언트 쪽 CChatClientSession과 스트레스 테스트의 CChatStressSession도 같은 ClocpSession을 상속한다.

세션 내부 경로

ClocpSession 안에서 수신은 CRingBuffer로, 송신은 CSendBuffer 묶음으로 갈라진다. 세션당 Recv와 Send는 각각 1개씩만 진행 중이고, 모든 I/O 요청은 ClocpEvent(OVERLAPPED)로 게시되며 이벤트의 owner가 완료 통지가 올 때까지 세션을 살려 둔다.

수신은 링버퍼로, 송신은 청크 버퍼 묶음으로 나간다



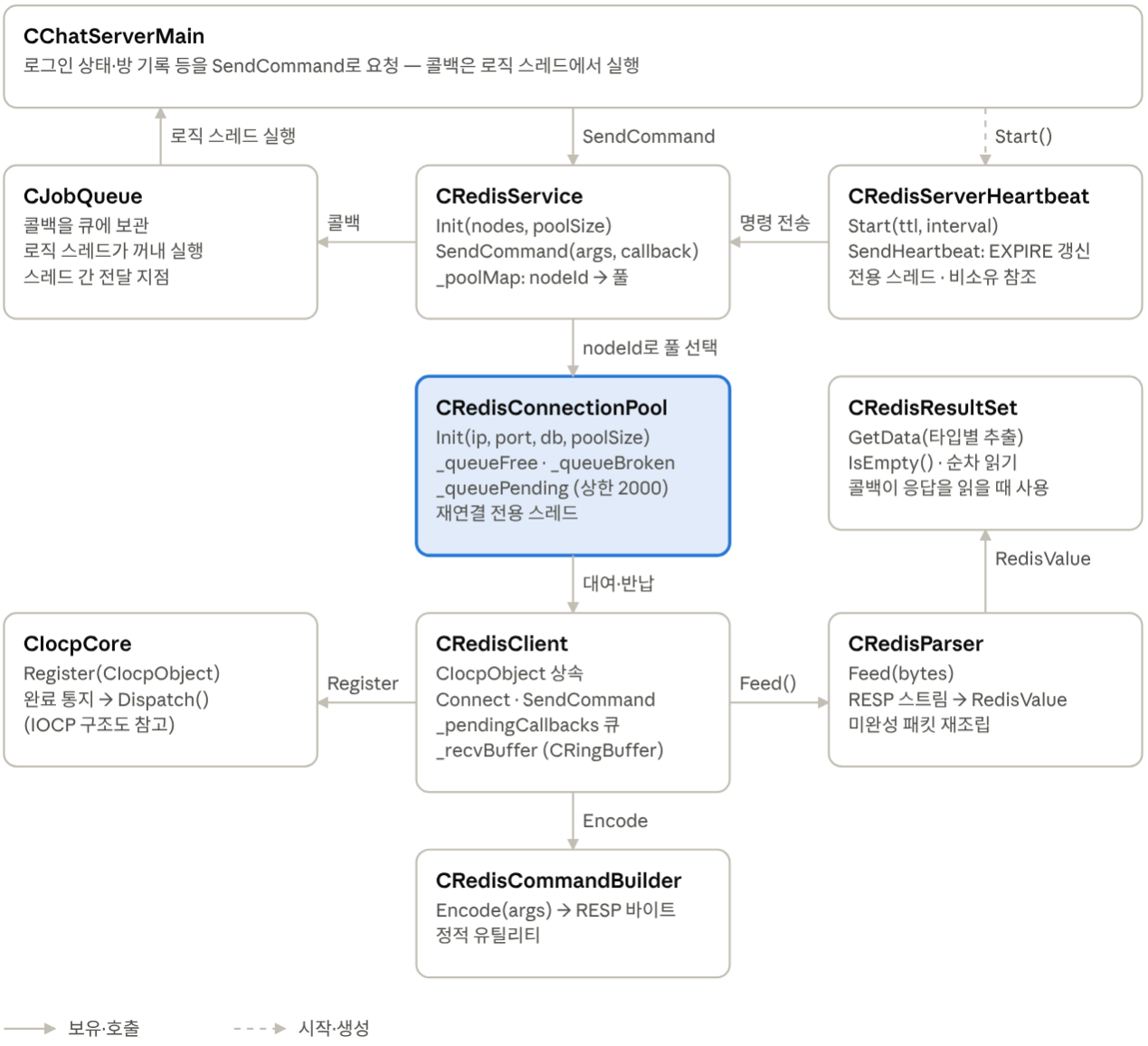
세션 내부 경로 · 수신 링버퍼, 송신 청크 버퍼, 이벤트

수신은 링버퍼에 쌓인 데이터를 CChatSession의 OnRecv가 헤더(size, type) 기준으로 완성된 패킷만 잘라 CPacketDispatcher로 넘긴다. 송신은 CSendBufferManager가 스레드별 청크(8KB)에서 구간을 열어 CSendBuffer로 주고, 세션이 _sendQueue에 모아 scatter-gather로 한 번에 WSASend한다.

Redis 클래스 구조

Redis 계층도 IOCP 위에 얹혀 있다. CRedisClient가 ClocpObject를 구현해 연결 1개를 IOCP 코어에 등록하고, CRedisConnectionPool이 노드별로 이 클라이언트들을 빌려주고 되돌려 받는다.

명령은 노드별 풀에서 빌린 IOCP 클라이언트로 나가고, 응답 콜백은 JobQueue로 돌아온다



Redis 클래스 구조 · 서비스·연결 풀·클라이언트·파서·JobQueue

명령은 CRedisCommandBuilder가 RESP 바이트로 만들어 보내고, 응답은 CRedisParser가 RedisValue로 조립한다. 풀의 연결이 모두 사용 중이면 요청은 대기 큐(상한 2000)에 쌓이고, 상한을 넘기면 SendCommand가 false를 반환한다. 그래서 처리량은 대체로 풀 크기 ÷ 왕복 시간으로 정해진다.

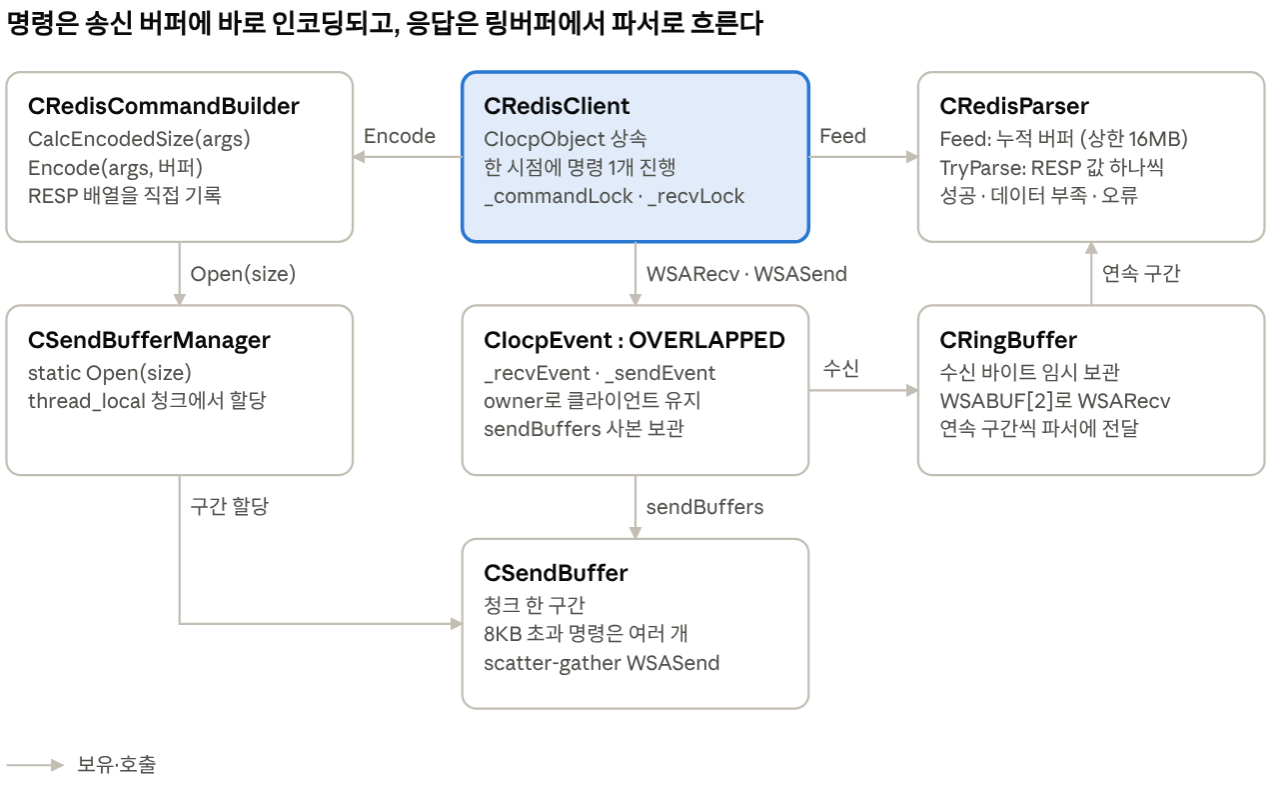
연결 풀 동작

- **대여·반납** — 커넥션 하나는 한 시점에 명령 하나만 진행하며, 응답 콜백이 끝나야 풀로 돌아온다. 돌아온 커넥션이 살아 있으면 대기 큐의 요청이 먼저 받아 가고, 대기 요청이 없으면 Free 큐로, 끊겨 있으면 격리 큐로 들어간다. Free 큐에서 꺼낼 때도 끊긴 커넥션은 격리 큐로 보낸다.

- **재연결** — 전용 스레드가 주기(기본 3초)마다 격리 큐를 훑는다. 클라이언트별 지수 백오프(500ms에서 시작해 최대 30초, 지터 최대 250ms)로 재시도 시점을 정하고, 이전 연결의 I/O 완료 통지가 모두 도착한 뒤에만 같은 OVERLAPPED를 재사용해 다시 연결한다.
- **세대 번호** — Clear()와 재Init()마다 증가한다. 이전 세대에서 빌려 간 커넥션의 늦은 콜백은 세대가 달라 풀 상태에 반영되지 않는다.
- **콜백 계약** — 풀이 접수한 명령(true 반환)의 콜백은 응답 또는 에러 값(연결 끊김, 전송 실패 등)으로 정확히 한 번 호출된다. 사용자 콜백이 예외를 던져도 커넥션 반납은 보장된다. 풀을 Clear()하면 대기 큐에 남은 요청은 폐기된다.
- **노드 구성** — CRedisService는 노드마다 풀을 하나씩 만들어 노드 ID로 관리하고(CClusterSpinUnorderedMap), 노드를 지정하지 않으면 첫 번째 노드를 쓴다. 모든 노드 풀이 연결된 경우에만 초기화가 성공하며, 응답 콜백은 CJobQueue를 거쳐 로직 스레드에서 실행된다.

클라이언트 내부 경로

CRedisClient 하나는 소켓 하나를 소유한 ClocpObject다. 접속은 논블로킹 connect와 select로 시간 제한(기본 3초)을 걸고, TCP keepalive(30초 유휴 후 10초 간격)를 켠 뒤 IOCP에 등록하고 첫 WSARecv를 게시한다. 논리 DB가 0이 아니면 SELECT까지 마쳐야 접속이 성공한다.



Redis 클라이언트 내부 경로 · 빌더, 송신 버퍼, 이벤트, 링버퍼, 파서

송신은 CRedisCommandBuilder가 크기를 먼저 계산하고, CSendBufferManager에서 연 버퍼에 RESP를 직접 기록한다(중간 std::string 복사 없음). 명령이 청크 크기(8KB)를 넘으면 여러 CSendBuffer로 나눠 담아 scatter-gather로 한 번에 보내고, 부분 전송은 남은 구간부터 이어 보낸다.

수신은 링버퍼의 연속 구간을 CRedisParser에 순서대로 넘기며, 파서는 미완성 데이터를 자체 누적 버퍼에 보관한다. 파서 누적 상한(16MB) 초과나 RESP 형식 오류가 나면 연결을 끊고, 대기 중이던 콜백은 사유별 에러 문자열로 완료된다. 파서는 배열 원소 수(4096)와 중첩 깊이(64)에도 상한을 둔다.

응답 읽기

CRedisResultSet은 RedisValue를 문자열 목록으로 펼친 뒤 GetData()로 읽기 커서를 따라 INT32, INT64, std::string, TCHAR 등 원하는 타입으로 순서대로 꺼낸다. 변환에 실패하면 커서는 그 자리에 남는다. 에러 응답과 빈 결과는 둘 다 IsEmpty()가 true이므로, 둘을 구분해야 하면 RedisValue::eType을 직접 확인한다.

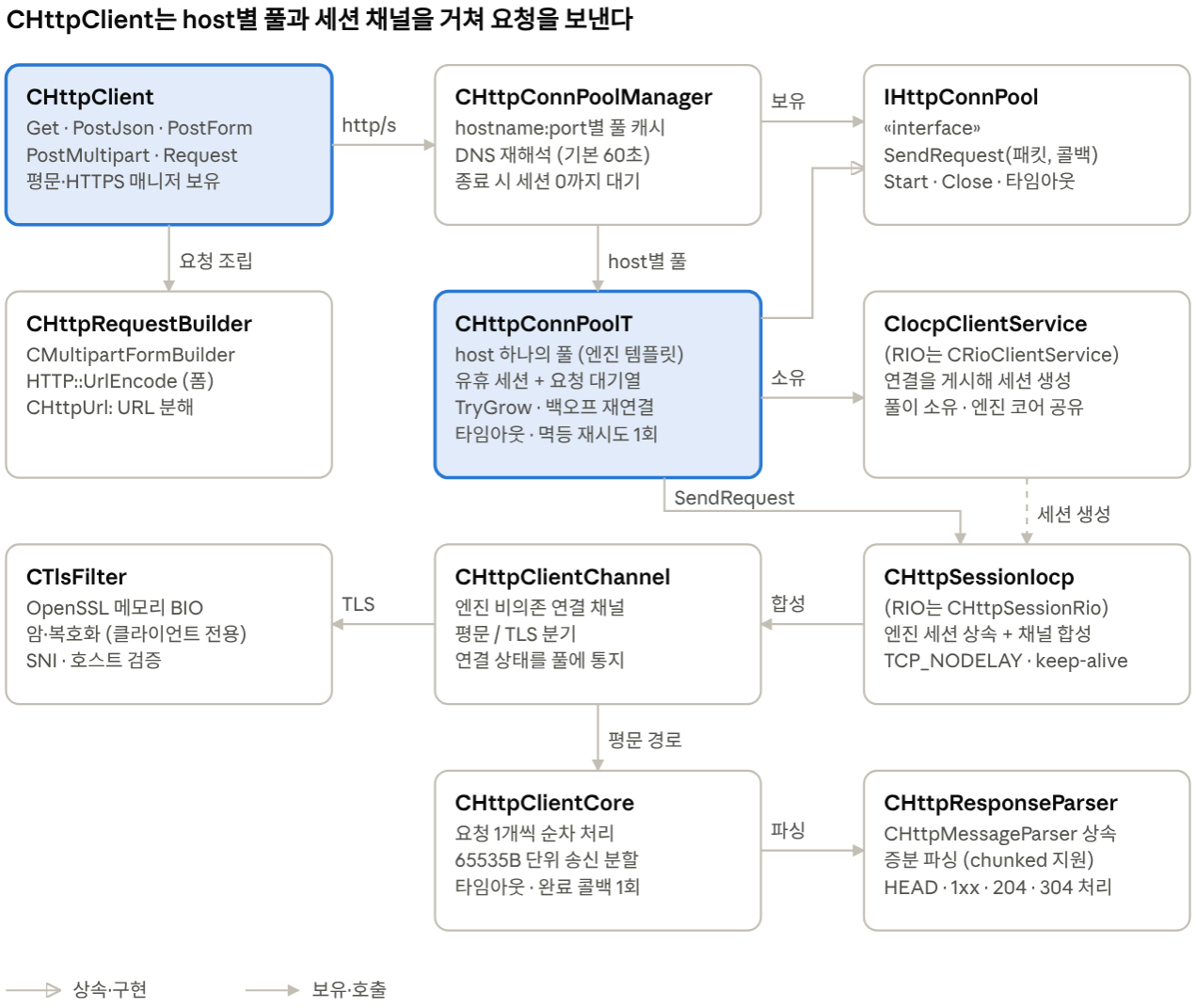
서버 하트비트

CRedisServerHeartbeat는 서버 시작 시 `{serverName}:{serverGroupId}:{serverChannelId}` 키에 Hash로 등록 정보(포트, pid, 세션 수, 시작·갱신 시각 등)를 HSET하고 TTL(기본 15초)을 건다. 전용 스레드가 주기(기본 5초)마다 등록 필드 전체를 다시 쓰고 EXPIRE로 TTL을 갱신하므로, Redis가 재시작돼 키가 사라져도 완전한 등록 정보로 복구된다. 비정상 종료하면 TTL 만료로 항목이 자동 소멸하고, 정상 종료(Stop)는 DEL로 즉시 지운다. 갱신 주기는 TTL보다 짧아야 하며, 세션 수는 등록한 콜백(SetSessionCountProvider)이 알려 준다.

HTTP 계층 구조

UtilCore의 Network/HTTP는 엔진(IOCP/RIO)에 의존하지 않는 HTTP/1.x 구현이다. 클라이언트 쪽(CHttpClient, 커넥션 풀, TLS)과 서버 쪽(요청 파서, 응답 빌더, 멀티파트, Range)이 같은 파서·빌더 기반을 공유한다. 파일 서버는 서버 쪽 클래스를 사용한다(파일 서버 구조 참고).

클라이언트 구조



HTTP 클라이언트 구조

- CHttpClient: URL의 scheme으로 평문/HTTPS 매니저를 골라 요청을 보낸다. SSL_CTX 없이 만들면 https 요청은 즉시 실패(success=false)로 통지된다.
- CHttpConnPoolManager: host(hostname:port)별 풀을 지연 생성해 캐싱한다. 같은 host로 첫 요청이 동시에 오면 한 스레드만 DNS 해석과 풀 생성을 맡는다. 주소는 기본 60초마다 다시 해석하고, 종료 시에는 모든 세션이 정리될 때까지 기다릴 수 있다.
- CHttpConnPoolT: host 하나의 풀이며, 세션·서비스 타입을 템플릿 파라미터로 받아 IOCP와 RIO가 같은 코드를 쓴다. 초기에는 minIdle개만 연결하고, 유휴 세션이 없어 요청이 대기열에 쌓이면 maxConnections까지 늘린다. 끊긴 연결은 지수 백오프(200ms~10초)로 다시 채우고, 멱등 요청은 1회 자동 재시도하며, 무응답 타임아웃 기본값은 30초다.
- CHttpSessionIocp / CHttpSessionRio: 엔진 세션의 훅을 CHttpClientChannel로 이어 주는 얇은 어댑터다. 채널은 평문이면 CHttpClientCore로 바이트를 직접 전달하고, HTTPS면 CTlsFilter(OpenSSL 메모리 BIO)로 암·복호화한 뒤 전달한다.

- CHttpClientCore: 커넥션 하나에서 요청을 한 번에 하나씩만 처리한다(파이프라이닝 없음). 완료 콜백은 정확히 한 번, 락 밖에서 호출한다.

메시지 파서와 빌더

HTTP 메시지 파서·빌더 구조

- 파서: CHttpMessageParser가 헤더·Content-Length·chunked·trailer 처리와 줄·헤더·본문 상한을 담당하고, 요청/응답 파서는 시작 줄과 본문 길이 규칙만 구현한다. 헤더 이름 앞뒤 공백, 서로 다른 Content-Length 중복처럼 메시지 경계가 모호해지는 입력은 모두 Error로 처리한다.
- 대용량 본문: CHttpRequestParser는 기본적으로 본문을 메모리에 누적하되 64MB를 넘으면 Error로 처리한다. 헤더 완료 콜백에서 본문 스트림 콜백을 걸면 본문을 누적하지 않고 흘려보내며, 요청별로 상한을 따로 줄 수 있다. 파일 서버의 업로드가 이 경로를 쓴다.
- 멀티파트: 소량 폼은 MultipartFormParser가 완성된 본문을 한 번에 해석하고, 대용량은 CMultipartStreamParser가 boundary 기준으로 파트 시작·데이터·끝 콜백을 보낸다(파트 헤더만 버퍼링).
- 빌더: char 바이트로만 조립하고 헤더는 flat vector로 보관하며, Reset으로 버퍼를 재사용한다. 요청 빌더와 응답 빌더는 CHttpBuilderBase를 공유한다.
- Range: 단일 바이트 구간만 지원한다. 다중 구간 요청은 무시하고 전체를 200으로 응답한다.

파일 서버 구조

파일 서버는 채팅 서버와 별개 프로세스로 동작하는 HTTP 서버다. 네트워크 계층(ClocpCore, ClocpServerService)과 Redis 계층(CRedisService), CJobQueue를 그대로 재사용하고, 세션만 HTTP를 해석하는 CFileServerSession으로 바꿨다. 채팅 서버와는 서로 직접 통신하지 않는다. 두 서버가 같은 Redis를 바라보며 업로드 토큰과 삭제 예약 목록을 주고받고, 클라이언트는 채팅 서버에서 받은 주소와 토큰으로 파일 서버에 직접 HTTP 요청을 보낸다.

파일 서버 클래스 구조

업로드 흐름

- 요청은 멀티파트로 받으며 본문을 메모리에 모으지 않고 임시 파일(upload_{랜덤 hex}.tmp)에 스트리밍으로 기록한다. 허용 크기는 MaxUploadBytes로 제한한다.
- 채팅 서버가 Redis에 발급한 일회용 토큰(UploadToken:{token})을 GET 후 DEL로 소비한다. 토큰이 없거나 이미 사용되었으면 거부한다.
- 검증이 끝나면 저장소로 옮기고 메타데이터를 등록한 뒤 PublicBaseUrl 기반의 접근 URL을 응답한다.

- 상태 코드: 잘못된 요청 400, 토큰 오류 401, 크기 초과 413, 저장 실패 500, 성공 200.

다운로드 흐름

- Range 요청을 지원한다. 구간이 유효하면 206, 범위를 벗어나면 416으로 응답한다.
- 본문은 64KB 풀 버퍼(CFileSendBufferPool)를 청크 단위로 재사용해 전송하므로 큰 파일도 메모리를 크게 쓰지 않는다.
- Content-Type은 파일 확장자로 추정한다.

삭제와 예약 삭제

- DELETE 요청은 파일과 메타데이터를 함께 제거한다.
- 채팅 서버가 Redis의 FileServer:PendingDeletions 목록에 삭제 대상을 넣으면, 파일 서버가 5초 주기로 LPOP 폴링해 저장소에서 파일을 지우고 메타데이터도 함께 정리한다. 목록에 항목이 남아 있으면 대기 없이 연속으로 비운다.
- DELETE 요청 자체에는 인증이 없다. 외부 노출 시에는 별도 보호가 필요하다.

저장소와 메타데이터

- IFileStorage 인터페이스 뒤에 CLocalFileStorage가 있다. 저장 경로는 {baseDir}/{ownerId}/{랜덤 hex}{확장자}이며, 경로에 ".."가 포함되면 읽기·삭제·스트림 열기 모두 거부한다.
- 저장소를 인터페이스로 분리했으므로 다른 백엔드로 교체해도 핸들러는 바뀌지 않는다.
- 파일 메타데이터(소유자·크기·타입)는 CFileMetadataRepository가 메모리에 보관한다.

채팅 서버와의 연동 (Redis)

- 업로드 토큰: 클라이언트가 채팅 서버에 토큰을 요청하면, 채팅 서버가 무작위 32바이트(16진 64자)를 만들어 `SET UploadToken:{token} {계정 public_id} EX 60` 으로 Redis에 넣는다. 응답에는 토큰과 파일 서버 주소(FileServerUrl)가 실린다. 파일 서버는 업로드 요청이 올 때 이 키를 조회해 소비하고, 값으로 소유 계정을 알아낸다. TTL이 60초라 발급 후 곧바로 쓰지 않으면 만료된다.
- 삭제 예약: 채팅 서버가 프로필 이미지를 교체하거나 지울 때, 지워진 URL이 자기 파일 서버 소유(`{FileServerUrl}/images/...`)이면 상대 경로만 `RPUSH FileServer:PendingDeletions` 로 넣는다. 외부 URL은 대상에서 제외한다.
- 주소 일치: 위 접두사 판별이 파일 서버가 만드는 공개 URL과 같은 형식이어야 하므로, 채팅 서버의 FileServerUrl과 파일 서버의 PublicBaseUrl은 같은 주소로 맞춘다.
- 클라이언트 경로: 채팅 서버는 파일 바이트를 중계하지 않는다. 업로드와 다운로드는 클라이언트가 파일 서버로 직접 보낸다.

채팅 서버 설정 (chatserver_config_mysql.json)

채팅 서버는 실행 폴더의 chatserver_config_mysql.json을 읽는다. 파일 서버와 맞물리는 항목은 FileServerUrl과 RedisNode다.

항목	설명	기본값
Name / GroupId / ChannelId	서버 이름과 그룹·채널 식별자	ChatServer / 1 / 1
IP / Port	수신 주소와 포트	127.0.0.1 / 30201
KeepAliveSec	세션 유지 시간(초)	3000
MaxSessionCount	최대 동시 세션 수	2000
WorkerThreadCnt	IOCP 워커 스레드 수	0
RedisPoolSize	노드당 Redis 연결 수	8
DbWorkerThreadCnt	DB 비동기 워커 스레드 수	4
HeartbeatTtlSec / HeartbeatIntervalSec	서버 하트비트 TTL과 갱신 주기(초)	15 / 5
FileServerUrl	클라이언트에 알려 줄 파일 서버 주소. 파일 서버의 PublicBaseUrl과 일치해야 한다	http://127.0.0.1:8081
MaxRoomsPerOwner	계정당 만들 수 있는 방 수 상한	3
DBNode	접속할 DB 노드(ODBC 드라이버, 주소, DB 이름 등)	MySQL, 127.0.0.1:3306, chat
RedisNode	접속할 Redis 노드. 파일 서버와 같은 Redis를 가리켜야 한다	1, 127.0.0.1:16379, 0

파일 서버 설정 (fileserver_config_mysql.json)

항목	설명	기본값
Name / GroupId / ChannelId	서버 이름과 그룹·채널 식별자	FileServer / 1 / 0
IP / Port	수신 주소와 포트	0.0.0.0 / 8081
MaxSessionCount	최대 동시 세션 수	500
WorkerThreadCnt	IOCP 워커 스레드 수	4
RedisPoolSize	노드당 Redis 연결 수	4

항목	설명	기본값
StorageDir	파일 저장 기준 디렉터리	ProfileImages
PublicBaseUrl	클라이언트에 알려 줄 접근 URL의 기준	http://127.0.0.1:8081
MaxUploadBytes	업로드 최대 크기 (바이트)	5368709120 (5GB)
RedisNode	접속할 Redis 노드(ID, 주소, DBIndex)	1, 127.0.0.1:16379, 0

기술 스택

영역	내용
서버 언어	C++, IOCP 기반 비동기 네트워크 엔진
클라이언트 언어	C#, WinForms
네트워킹	TCP 커스텀 바이너리 프로토콜, 고정 헤더 + 패킷 타입
영구 저장소	MySQL (ODBC) — 계정, 방, 채팅 메시지, 방 참여 이력
캐시·실시간 데이터	Redis — 로그인 상태, 온라인 카운터, 방 멤버 집계
파일 전송	별도 파일 서버(HTTP) — 첨부파일·프로필 이미지 업로드/다운로드

설계 원칙

- **비동기·논블로킹 우선** — 네트워크 I/O와 DB 요청 모두 콜백 기반 비동기로 처리하고, 로직 스레드는 큐를 통해서만 결과를 받는다.
- **프로토콜의 일관된 확장 패턴** — 목록형 요청은 요청 ID, 페이지 번호, 페이지 크기를 공통으로 쓰고, 항목 단건 응답을 스트리밍한 뒤 완료 응답으로 마무리하는 패턴을 방 목록·방 멤버 목록 등에 동일하게 적용한다.
- **클라이언트는 서버 상태를 그대로 반영** — 즉시 반응이 필요한 일부(프로필 이미지 등)만 낙관적으로 먼저 보여주고, 서버 응답으로 확정·정정한다.